

Manual Técnico - PICTOSEQ

Título: Sistema Web para Generación de Secuencias de Pictogramas

Autor: Azucena Abigail Vargas Hernández

Versión: 1.0

Fecha: 01/Diciembre/2025

Tabla de contenido

Introducción.....	4
1.1 Visión General.....	4
2. Configuración y Arranque.....	4
2.1 pictoweb/__init__.py - Núcleo de la Aplicación Flask	4
2.2 pictoweb/config.py – Configuración Global	5
3. Controlador de Rutas y Lógica Principal pictoweb/routes.py	7
3.1 Visión general	7
3.2 Estructura del código	7
3.3 Funciones de Utilidad	8
3.4 Rutas de Navegación Básica	8
3.5 Sistema de Categorías	9
3.6 Funcion de Empaquetado _zip_categoria	11
3.7 Creación de Nuevas Secuencias.....	12
3.8 APIs para Interactividad	13
4.Gestión de Acceso a Datos con Cacheo	15
4.1 Visión General.....	16
4.2 Arquitectura del Módulo.....	16
4.3 Funciones Principales.....	16
4.4 Estrategia de Cacheo.....	18
5.Sistema de Personalización Persistente pictoweb/overrides.py.....	19
5.1 Visión general	19
5.2 Funciones Principales.....	19
5.3 Integración con la Aplicación	21
5.4 Flujo de Datos	22
6.Fachada para el Servicio BETO – pictoweb/beto_service.py	22
6.1 Vision general	22
6.2 Funciones Principales.....	23
6.3 Integración con la Aplicación	24
6.4 Flujo de Datos	25
6.5 Dependencias	25
7.Referencia Externa – pictoweb/beto_local.py.....	25
7.1 Visión General.....	25
7.2 Puntos de Integración con la Aplicación.....	26
7.3 Referencia Cruzada	27

8.Fachada para el Servicio CLIP – pictoweb/clip_service.py	27
8.1 Visión General.....	27
8.2 Integración con la Aplicación	27
8.3 Flujo de Procesamiento CLIP.....	28
8.4 Casos de Uso	28
9.Referencia Externa – pictoweb/clip_local.py	28
9.1 Visión General.....	28
10. Templates del sistema.....	29
10.1 base.html – Layout Principal.....	29
10.2 home.html – Página de Inicio	29
10.3 acerca_de.html – Página “Acerca de”	29
10.4 manual_usuario.html – Página “Manual de Usuario”	30
10.5 categoria.html – Hub de Categorías.....	31
10.6 categoria_detalle.html – Detalle de Categoría con Carrusel.....	32
10.7 crear_picto.html – Creación de Secuencias Personalizadas	33
11. Consideraciones finales del sistema	34

Introducción

1.1 Visión General

PICTOSEQ es una aplicación web desarrollada con **Flask** que permite generar secuencias personalizadas de pictogramas utilizando modelos de IA (BETO y CLIP) para la búsqueda semántica y visual.

2. Configuración y Arranque

2.1 pictoweb/__init__.py - Núcleo de la Aplicación Flask

Propósito: define la función `create_app()` que configura e inicializa la aplicación Flask, registra los blueprints (rutas) y establece las configuraciones por defecto para los servicios de IA.

```
from flask import Flask
from .config import Config
from .routes import bp as main_bp

def create_app():
    app = Flask(__name__)
    app.config.from_object(Config)
    app.register_blueprint(main_bp)
    app.config.setdefault("PICTOS_DIR", r"C:\Users\Anabelen\Desktop\NOVENO SEMESTRE\PICTOGRAMAS\PICTOGRAMAS\pictogramas")
    # CLIP
    app.config.setdefault("CLIP_TEXT_MODEL", "sentence-transformers/clip-ViT-B-32-multilingual-v1")
    app.config.setdefault("CLIP_IMAGE_MODEL", "clip-ViT-B-32")
    app.config.setdefault("CLIP_FORCE_CPU", False)
    # Si algún día se expone servicio remoto:
    app.config.setdefault("CLIP_REMOTE_URL", None)
    return app
```

Ilustración 1 __init__.py__

Flujo de Inicialización

1. **Instanciación Flask:** Crea el objeto aplicación con `__name__` para resolver rutas relativas.
2. **Carga Configuración:** Importa todas las variables definidas en `Config`.
3. **Registro de Rutas:** Conecta todas las rutas definidas en `routes.py` mediante el Blueprint.
4. **Configuración Específica:** Establece valores por defecto para directorios y modelos IA.
5. **Retorno:** Devuelve la aplicación lista para ser ejecutada por `run.py`.

Dependencias

- Flask: Framework web principal

- `.config.Config`: Clase de configuración personalizada
- `.routes.bp`: Blueprint con todas las rutas de la aplicación

Consideraciones Técnicas

- **Patrón Factory**: Usa el patrón de fábrica de aplicaciones de Flask para una creación flexible.
- **Configuración Escalonada**: Los `setdefault()` permiten sobrescribir valores desde variables de entorno.
- **Extensibilidad**: La estructura facilita añadir blueprints adicionales (API, admin, etc.).

2.2 pictoweb/config.py – Configuración Global

Propósito: Define la clase `Config` que centraliza todos los parámetros de configuración: rutas de archivos, modelos de IA, directorios de datos y opciones del sistema.

```
import os
class Config:

    BETO_INFERENCE_MODE = "local" # forzamos local
    BETO_MODEL_NAME      = "dccuchile/bert-base-spanish-wwm-cased"
    BETO_FORCE_CPU       = False   # True si queremos obligar CPU
    # cache de embeddings - se crea la primera vez
    BETO_CACHE_EMB       = r"C:\Users\Anabelen\Desktop\NOVENO SEMESTRE\PICTOGRAMAS\PICTOGRAMAS\cache\emb_pictos_cache.npy"

    PERSIST_OVERRIDES = False # No aplicar cambios guardados al recargar

    SECRET_KEY = os.environ.get('SECRET_KEY', 'dev-key-change-me')
    BOOTSTRAP_DIR = r"C:\Users\Anabelen\Desktop\NOVENO SEMESTRE\PICTOGRAMAS\PICTOGRAMAS\bootstrap-5.3.8-dist"
    PICTOS_DIR    = r"C:\Users\Anabelen\Desktop\NOVENO SEMESTRE\PICTOGRAMAS\PICTOGRAMAS\pictogramas"
    SECUENCIAS_DIR = r"C:\Users\Anabelen\Desktop\NOVENO SEMESTRE\PICTOGRAMAS\PICTOGRAMAS\Secuencias"
    CANDIDATOS_DIR = r"C:\Users\Anabelen\Desktop\NOVENO SEMESTRE\PICTOGRAMAS\PICTOGRAMAS\candidatos"
    SELECCIONES={
        'salud':os.path.join(SECUENCIAS_DIR,'selecciones_salud_final.json'),
        'educacion':os.path.join(SECUENCIAS_DIR,'selecciones_educacion_final.json'),
        'cultura': os.path.join(SECUENCIAS_DIR,'selecciones_cultura_final.json')
    }
    CANDIDATOS={
        'salud':os.path.join(CANDIDATOS_DIR,'candidatos_salud_final.json'),
        'educacion':os.path.join(CANDIDATOS_DIR,'candidatos_educacion_final.json'),
        'cultura':os.path.join(CANDIDATOS_DIR,'candidatos_cultura_final.json'),
    }
```

Ilustración 2 config.py

Estructura de Configuración

Sección	Propósito	Variables Clave
BETO	Modelo de embeddings en español	BETO_MODEL_NAME, BETO_CACHE_EMB
Overrides	Personalización por categoría	PERSIST_OVERRIDES
Seguridad	Claves y autenticación	SECRET_KEY
Rutas	Directorios del sistema	PICTOS_DIR, SECUENCIAS_DIR
Datos	Archivos JSON por categoría	SELECCIONES, CANDIDATOS

Configuraciones Clave Explicadas

BETO Configuration:

- BETO_INFERENCE_MODE = "local": Siempre usa el modelo local (sin API externa).
- BETO_MODEL_NAME: Modelo pre-entrenado en español de la Universidad de Chile.
- BETO_CACHE_EMB: Ruta a cache de embeddings para mejorar rendimiento.

Estructura de Datos:

- Tres categorías principales: salud, educación, cultura.
- Cada categoría tiene:
 - selecciones_[categoria]_final.json: Secuencias predefinidas.
 - candidatos_[categoria]_final.json: Pictogramas candidatos para búsqueda.

Rutas Absolutas:

- Usa rutas absolutas de Windows (ej: C:\Users\...).
- En producción, deberían usarse variables de entorno.

Dependencias

- os: Para manipulación de rutas y variables de entorno.

Consideraciones de Implementación

- **Entornos Diferentes:** SECRET_KEY usa os.environ.get() para permitir diferentes valores por entorno.
- **Rutas Específicas:** Las rutas están configuradas para el entorno de desarrollo actual.
- **Extensibilidad:** La estructura SELECCIONES y CANDIDATOS permite añadir nuevas categorías fácilmente.

3. Controlador de Rutas y Lógica Principal pictoweb/routes.py

3.1 Visión general

El archivo routes.py es el **controlador principal** de la aplicación Flask, definiendo todas las rutas, endpoints API y la lógica para la manipulación de secuencias de pictogramas. Implementa un patrón MVC donde las rutas actúan como controladores entre las vistas (templates) y los modelos (servicios de IA y datos).

3.2 Estructura del código

3.2.1 Importaciones y Dependencias:

```
import os, io, zipfile, json, re, unicodedata
from flask import (
    Blueprint, render_template, request, send_file, abort,
    current_app, jsonify
)
from .data_access import get_selecciones, map_candidatos_por_oracion
from .beto_service import infer_pictos_for_text
from .clip_service import assemble_with_clip
```

Ilustración 3 Importaciones y Dependencias routes.py

Componentes Importados:

- **Módulos estándar:** Manipulación de archivos, JSON, cadenas y Unicode
- **Componentes Flask:** Elementos esenciales para routing y respuestas HTTP
- **Servicios locales:** Acceso a datos y modelos de IA

3.2.2 Blueprint Principal:

```
bp = Blueprint("main", __name__)
```

Ilustración 4 Blueprint Principal - routes.py

- **Nombre:** "main" - Identificador único dentro de Flask

- **Registro:** Se registra en `create_app()` del archivo `__init__.py`

3.3 Funciones de Utilidad

3.3.1 slugify(s: str) -> str:

Propósito: Normaliza cadenas para usarlas en nombres de archivo/carpeta

```
def slugify(s: str) -> str:
    """Convierte un texto en un slug seguro para nombres de carpetas/archivos."""
    s = unicodedata.normalize("NFKD", s).encode("ascii", "ignore").decode("ascii")
    s = re.sub(r"[^a-zA-Z0-9\-\_\.\ ]+", "", s).strip().lower()
    return re.sub(r"[\s]+", "-", s)[:64] or "item"
```

Ilustración 5 Funcion slugify - routes.py

Proceso:

1. Normaliza Unicode a forma compatible ASCII
2. Elimina caracteres especiales
3. Convierte a minúsculas
4. Reemplaza espacios con guiones
5. Limita a 64 caracteres
6. Ejemplo: “Hola mundo” – “hola-mundo”

3.4 Rutas de Navegación Básica

3.4.1 Página de Inicio (/):

```
@bp.route("/")
def home():
    return render_template("home.html")
```

Ilustración 6 Página de inicio - routes.py

- **Template:** home.html
- **Propósito:** Punto de entrada principal de la aplicación

3.4.2 Página “Acerca de” (/acerca-de):

```
@bp.route("/acerca-de")
def acerca_de():
    return render_template("acerca_de.html")
```

Ilustración 7 Página Acerca de - routes.py

- **Contenido:** Información sobre el proyecto.

3.4.3 Pagina “Manual de Usuario” (/manual-usuario):

```
@bp.route("/manual-usuario")
def manual_usuario():
    return render_template("manual_usuario.html")
```

Ilustración 8 Pagina Manual de Usuario - routes.py

- **Recursos:** Enlace a PDF y PowerPoint en static/assets/manual_usuario/

3.5 Sistema de Categorías

3.5.1 Hub de Categorías (/categorías):

```
@bp.route("/categorias")
def categorias_hub():
    counts = {
        "salud": len(get_selecciones("salud") or []),
        "educacion": len(get_selecciones("educacion") or []) if current_app.config["SELECCIONES"].get("educacion") else 0,
        "cultura": len(get_selecciones("cultura") or []) if current_app.config["SELECCIONES"].get("cultura") else 0,
    }
    return render_template("categoria.html", counts=counts)
```

Ilustración 9 Hub de categorías - routes.py

Lógica:

- Calcula el número de secuencias por categoría
- Maneja categorías opcionales (verifica existencia en configuración)
- Pasa conteos al template para visualización

3.5.2 Detalle de Categoría (/categoría/<cat>):

```
@bp.route("/categoria/<cat>", methods=["GET", "POST"])
def categoria_detalle(cat):
    if cat not in {"salud", "educacion", "cultura"}:
        abort(404)
```

Ilustración 10 Detalle de Categoría - routes.py

Validación: Solo permite categorías predefinidas

Métodos:

- **GET:** Muestra la interfaz con carrusel y lista
- **POST:** Procesa descarga con overrides temporales

Flujo POST:

```
indices = [int(i) for i in request.form.getlist("indices") if i.isdigit()]
if not indices:
    indices = None

# 2) overrides de IMÁGENES (objeto dict {"0":[...], "3":[...]})
edits_json = request.form.get("edits_json") or "{}"
try:
    client_edits_imgs = json.loads(edits_json)
except Exception:
    client_edits_imgs = {}

# Normaliza a lista alineada con selecciones (None donde no hay override)
effective_for_zip = None
if isinstance(client_edits_imgs, dict) and client_edits_imgs:
    effective_for_zip = [None] * len(selecciones)
    for k, v in client_edits_imgs.items():
        try:
            i = int(k)
            if 0 <= i < len(selecciones) and isinstance(v, list):
                effective_for_zip[i] = [n for n in v if n] # filtra vacíos
        except Exception:
            continue

# 3) overrides de TOKENS/ORACIÓN (dict {"0":[...]} o {"0":"texto"})
edits_tokens_raw = request.form.get("edits_tokens_json") or "{}"
try:
    client_edits_tokens = json.loads(edits_tokens_raw)
except Exception:
    client_edits_tokens = {}

# 4) empaqueta y envía ZIP
return _zip_categoria(
    cat,
    selecciones,
    effective_for_zip,          # imágenes override por índice
    indices,
    token_overrides=client_edits_tokens # tokens/oración override por índice
)
```

Ilustración 11 Flujo POST - routes.py

3.5.3 Descarga Directa de Categoría (/descargar/categoría/<cat>):

```
@bp.route("/descargar/categoria/<cat>")
def descargar_categoria(cat):
    selecciones = get_selecciones(cat)
    if not selecciones:
        abort(404)
    return _zip_categoria(cat, selecciones, effective=None, indices=None)
```

Ilustración 12 Descarga de Categoría - routes.py

- **Sin modificaciones:** Descarga todas las secuencias en su estado original
- **Validación:** Verifica existencia de datos antes de proceder

3.6 Funcion de Empaquetado _zip_categoria

3.6.1 Propósito y Parámetros:

```
def _zip_categoria(cat, selecciones, effective, indices, token_overrides=None):
    """
    Empaqueta la categoría en ZIP.

    - selecciones: lista de dicts con claves: 'oracion', 'tokens', 'secuencia_final'/'elecciones'
    - effective:    lista de listas de imágenes (overrides de imágenes) o None
    - indices:      subconjunto de índices a incluir o None (todas)
    - token_overrides: dict { "idx_str": list[str] | str } con tokens/oración editados
    """
    token_overrides = token_overrides or {}
```

Ilustración 13 Funcion descarga zip de categorías - routes.py

Parámetros:

- cat: Categoría actual
- selecciones: Lista de secuencias originales
- effective: Overrides de imágenes por índice
- indices: Subconjunto de secuencias a incluir
- token_overrides: Modificaciones de tokens/oración

3.6.2 Estructura del ZIP Generado:

```

categoria_secuencias.zip
├─ salud/
│   └─ 000-visitar-al-medico/
│       └─ manifest.json
│           └─ imagenes/
│               └─ medico.jpg
│                   └─ hospital.jpg
│   └─ 001-tomar-medicamento/
└─ educacion/
    └─ ...

```

Ilustración 14 Estructura descarga del zip - routes.py

Contenido de manifest.json

```

{
"categoria": "salud",
"oracion": "visitar al médico",
"tokens": ["visitar", "al", "médico"],
"imagenes": ["medico.jpg", "hospital.jpg"],
"fuelle": "selecciones_originales"
}

```

3.7 Creación de Nuevas Secuencias

3.7.1 Pagina de Creación (/crear):

```

@bp.route("/crear", methods=["GET", "POST"])
def crear_picto_secuencia():
    raw_basket = request.form.get("basket_json") or request.args.get("basket_json") or ""
    try:
        saved_list = json.loads(raw_basket) if raw_basket else []
    except Exception:
        saved_list = []

    if request.method == "POST":
        frase = (request.form.get("frase") or "").strip()
        if not frase:
            return render_template("crear_picto.html", error="Escribe una frase.", saved_list=saved_list)
        try:
            result = infer_pictos_for_text(frase, dominio=None) # modo local
            tokens = result.get("tokens") or []
            sugerida = result.get("sugerida") or [None]*len(tokens)
            cand_map = result.get("candidatos_por_token") or {}
            return render_template(
                "crear_picto.html",
                frase=frase,
                tokens=tokens, sugerida=sugerida,
                candidatos_map=cand_map,
                saved_list=saved_list
            )
        except Exception as e:
            return render_template("crear_picto.html", error=f"No se pudo inferir: {e}", saved_list=saved_list)

    return render_template("crear_picto.html", saved_list=saved_list)

```

Ilustración 15 Funcion crear - routes.py

Flujo GET:

- Muestra formulario vacío
- Opcionalmente carga secuencias del "carrito" (basket_json)

Flujo POST:

1. Valida frase de entrada
2. Llama a `infer_pictos_for_text()` para obtener sugerencias BETO
3. Renderiza resultados con tokens, pictogramas sugeridos y candidatos

3.7.2 Descarga de Secuencias Creadas (/descargar/creadas_lote):

```
@bp.route("/descargar/creadas_lote", methods=["POST"])
def descargar_creadas_lote():
    raw = request.form.get("basket_json") or "[]"
    try:
        items = json.loads(raw)
    except Exception:
        items = []
    if not items:
        abort(400)
```

Ilustración 16 Descargar secuencias personalizadas - routes.py

- **Formato:** Similar a `_zip_categoria` pero para secuencias personalizadas
- **Estructura:** Carpeta creadas/ con subcarpetas numeradas

3.8 APIs para Interactividad

3.8.1 Re-inferencia Completa (/api/reinfer):

```
# --- API: re-inferir candidatos para una oración editada ---
@bp.post("/api/reinfer")
def api_reinfer():
    data = request.get_json(silent=True) or {}
    frase = (data.get("frase") or "").strip()
    dominio = (data.get("dominio") or "").strip() or None
    if not frase:
        return jsonify({"error": "frase vacía"}), 400
    try:
        result = infer_pictos_for_text(frase, dominio=dominio)
        # esperado: { tokens: [...], sugerida: [...], candidato: [...] }
        return jsonify(result)
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Ilustración 17 API: re-inferir candidatos para una oración editada - routes.py

Propósito: Cuando el usuario modifica una oración en tiempo real

Entrada: {"frase": "texto", "dominio": "salud"}

Salida: Misma estructura que infer_pictos_for_text()

3.8.2 Re-inferencia por Token (/api/reinfer_token):

```
@bp.post("/api/reinfer_token")
def api_reinfer_token():
    data = request.get_json(silent=True) or {}
    frase = (data.get("frase") or "").strip()
    pos = data.get("pos") # índice (int) del token editado según la UI
    raw_tok = (data.get("token") or "").strip()
    dominio = (data.get("dominio") or "").strip() or None
    if not frase or pos is None:
        return jsonify({"error": "faltan parámetros (frase, pos)"}), 400

    # normalizador simple sin tildes
    def _norm(s: str) -> str:
        s = unicodedata.normalize("NFKD", s)
        s = "".join(ch for ch in s if not unicodedata.combining(ch))
        return s.lower().strip()
```

Ilustración 18 Re-inferencia por token - routes.py

Caso de uso: Modificación de una sola palabra sin reprocesar toda la oración

Lógica de búsqueda:

1. Intenta usar índice posicional
2. Si falla, busca por texto normalizado (sin tildes, minúsculas)

3.8.3 Ensamblaje CLIP (/api/clip/assemble):

```
@bp.post("/api/clip/assemble")
def api_clip_assemble():
    try:
        payload = request.get_json(force=True)
        oracion = (payload or {}).get("oracion", "").strip()
        tokens = (payload or {}).get("tokens") or []
        candidatos = (payload or {}).get("candidatos") or {}

        if not oracion or not tokens or not isinstance(candidatos, dict):
            return jsonify({"ok": False, "error": "payload inválido"}), 400

        from .clip_service import assemble_clip
        result = assemble_clip(oracion, tokens, candidatos)
        return jsonify({"ok": True, "result": result})
    except Exception as e:
        return jsonify({"ok": False, "error": str(e)}), 500
```

Ilustración 19 Función para ensamblar CLIP - routes.py

Integración: Llama a `assemble_clip()` del servicio CLIP

3.9 Servicio de Archivos Estáticos:

```
@bp.route("/pictos/<path:filename>")
def pictos(filename):
    from flask import send_from_directory
    # Seguridad: sólo sirve archivos por basename dentro de PICTOS_DIR
    return send_from_directory(current_app.config["PICTOS_DIR"], os.path.basename(filename))
```

Ilustración 20 Servicio de Archivos - routes.py

Seguridad: Usa `os.path.basename()` para prevenir path traversal attacks

Propósito: Sirve imágenes de pictogramas desde el directorio configurado

3.10 Diagrama de Flujo de Rutas:

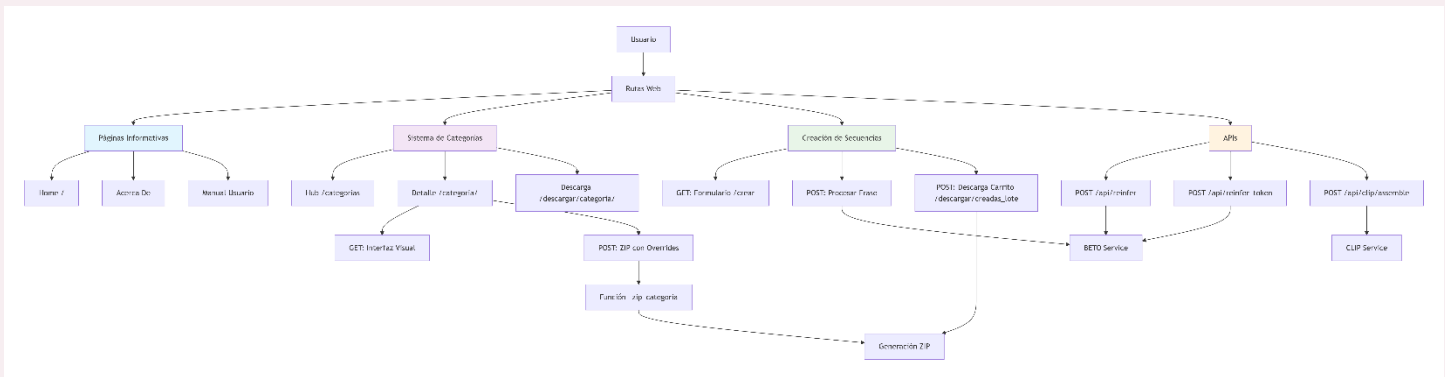


Ilustración 21 Diagrama de flujo de Rutas

4. Gestión de Acceso a Datos con Cacheo

```
import json
from functools import lru_cache
from flask import current_app

def _load_json(path):
    with open(path, 'r', encoding='utf-8') as f:
        return json.load(f)

@lru_cache(maxsize=16)
def get_selecciones(cat):
    path=current_app.config['SELECCIONES'].get(cat)
    if not path: return None
    return _load_json(path)

@lru_cache(maxsize=16)
def get_candidatos(cat):
    path=current_app.config['CANDIDATOS'].get(cat)
    if not path: return None
    return _load_json(path)

def map_candidatos_por_oracion(cat):
    candidatos=get_candidatos(cat)
    if not candidatos: return {}
    salida={}
    lista=candidatos.get('oraciones') if isinstance(candidatos,dict) else candidatos
    if not isinstance(lista,list): return {}
    for item in lista:
        key=item.get('oracion')
        if not key: continue
        salida[key]={ 'tokens': item.get('tokens',[]), 'candidatos_por_token': item.get('candidatos',{}) }
    return salida
```

Ilustración 22 Funciones de Cacheo

4.1 Visión General

El módulo `data_access.py` actúa como la **capa de acceso a datos** de la aplicación, proporcionando funciones para cargar y transformar los archivos JSON de secuencias y candidatos. Implementa un sistema de **cacheo LRU (Least Recently Used)** para minimizar las operaciones de I/O y mejorar significativamente el rendimiento en operaciones repetitivas.

4.2 Arquitectura del Módulo

4.2.1 Importaciones Esenciales

Propósito de cada importación:

- `json`: Para deserializar archivos JSON con codificación UTF-8
- `lru_cache`: Decorador para implementar caché de tamaño fijo con política LRU
- `current_app`: Para acceder a la configuración de la aplicación Flask sin import circular

4.2.2 Patrón de Diseño:

- **Repository Pattern**: Centraliza el acceso a datos en funciones específicas
- **Cache-Aside Pattern**: Carga datos en caché bajo demanda
- **Singleton implícito**: El caché es único por proceso de trabajo Flask

4.3 Funciones Principales

4.3.1 Funcion Auxiliar `_load_json(path)`:

Características:

- **Simplicidad**: Una única responsabilidad - leer JSON desde disco
- **Codificación**: UTF-8 para soportar caracteres en español (tildes, eñes)
- **Sin manejo de errores**: Las excepciones se propagan al llamador

Consideraciones de Seguridad:

- No valida el contenido del JSON
- Asume que las rutas son seguras (validadas en niveles superiores)

4.3.2 Funcion Cacheada `get_selecciones(cat)`:

Parámetros:

- **cat:** String con la categoría ('salud', 'educacion', 'cultura')

Comportamiento del Cacheo:

- **Tamaño máximo:** 16 entradas en caché
- **Política LRU:** Las entradas menos usadas se eliminan primero
- **Clave de caché:** Combinación de nombre función + argumento (cat)

Flujo de Ejecución:

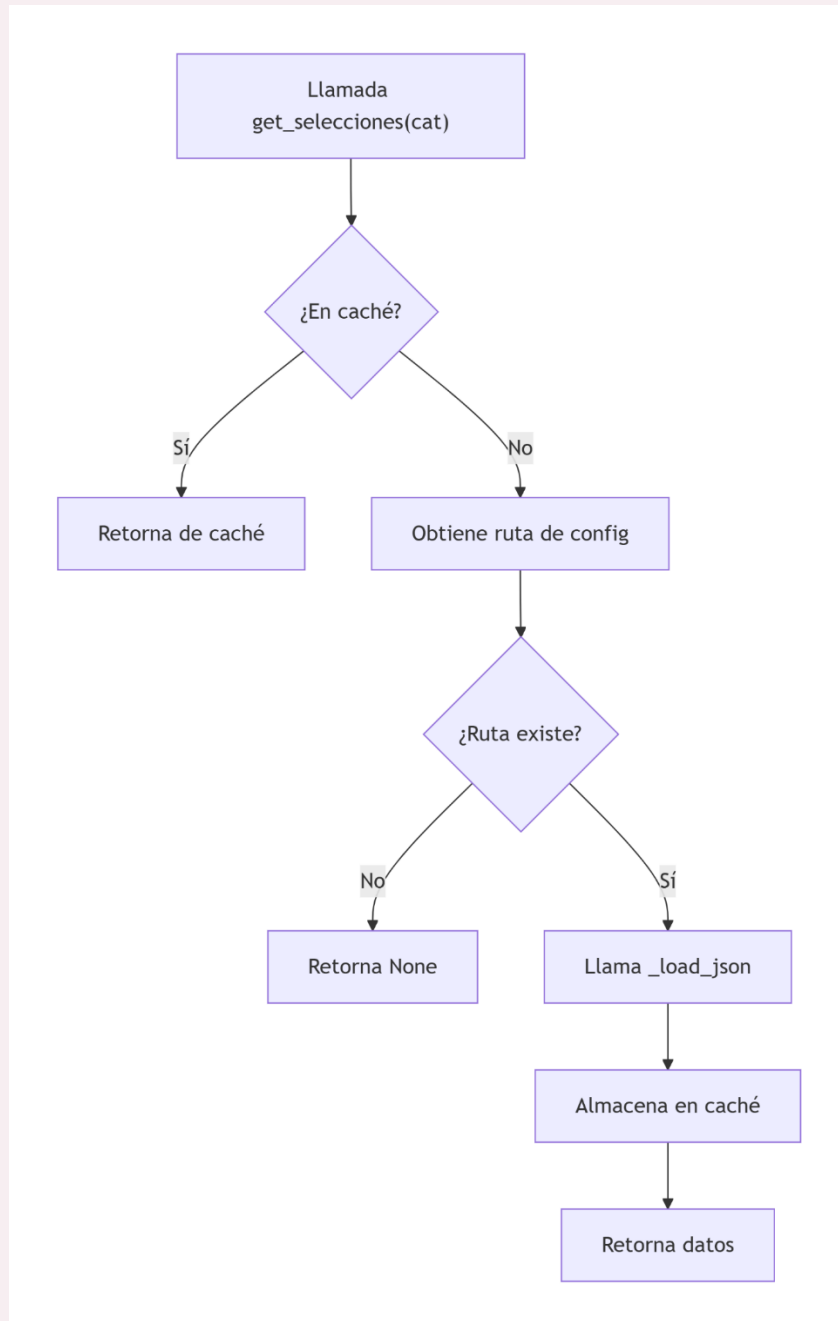


Ilustración 23 Flujo de ejecución - routes.py

4.3.3 Funcion Cacheada `get_candidatos(cat)`:

Simetría con `get_selecciones`:

- Misma estructura y lógica de cacheo
- Diferente clave en configuración (CANDIDATOS vs SELECCIONES)
- Usada principalmente por `map_candidatos_por_oracion`

4.3.4 Funcion de Transformacion `map_candidatos_por_oracion(cat)`:

Propósito: Transforma la estructura de datos de candidatos para acceso eficiente

Lógica de Transformación:

1. **Flexibilidad estructural:** Acepta tanto lista directa como diccionario con clave 'oraciones'
2. **Validación robusta:** Verifica tipos en cada paso (`isinstance`)
3. **Limpieza de datos:** Omite elementos sin clave 'oracion'
4. **Valores por defecto:** Usa listas/diccionarios vacíos para claves faltantes

4.4 Estrategia de Cacheo

4.4.1 Configuración LRU:

Análisis del tamaño:

- **3 categorías actuales × 2 funciones** = 6 entradas máximas
- **Tamaño 16:** Permite expansión futura (nuevas categorías, archivos)
- **Overhead mínimo:** Cada entrada en caché es una referencia a la estructura de datos

4.4.2 Características del Cacheo:

- **Por proceso:** El caché vive mientras viva el proceso de trabajo Flask
- **Thread-safe:** `lru_cache` es seguro para uso concurrente en Python
- **Invalida automáticamente:** Cuando el caché llega a 16 entradas, elimina la menos usada

5. Sistema de Personalización Persistente pictoweb/overrides.py

5.1 Visión general

El módulo overrides.py gestiona las **sobreescrituras (overrides)** persistentes por categoría. Permite a los usuarios modificar las secuencias de pictogramas (imágenes y tokens) y guardar esos cambios en archivos JSON específicos por categoría, ubicados en el directorio overrides/.

5.2 Funciones Principales

5.2.1

```
import os, json
from flask import current_app

def _path_for(cat: str):
    # Un archivo por categoría, p.ej.: overrides/salud.overrides.json
    base = current_app.config.setdefault("OVERRIDES_DIR",
    |   |   | os.path.join(os.path.dirname(__file__), "overrides"))
    os.makedirs(base, exist_ok=True)
    return os.path.join(base, f"{cat}.overrides.json")
```

Ilustración 24 Función: `_path_for(cat: str)` - overrides.py

Propósito: Construye la ruta al archivo de overrides para una categoría dada.

Comportamiento:

- Utiliza el directorio configurado en OVERRIDES_DIR o, por defecto, pictoweb/overrides/.
- Crea el directorio si no existe (os.makedirs con exist_ok=True).
- El archivo se nombra como {categoria}.overrides.json.

5.2.2

```
def load_overrides(cat: str) -> dict:
    p = _path_for(cat)
    if not os.path.isfile(p): return {}
    with open(p, "r", encoding="utf-8") as f:
        return json.load(f) # esperado: {"0":[...], "1":[...]}
```

Ilustración 25 Función: `load_overrides(cat: str) -> dict` - overrides.py

Propósito: Carga los overrides existentes para una categoría.

Retorno: Un diccionario donde las claves son índices (en string) de las oraciones y los valores son las listas de imágenes sobreescritas.

Nota: Si el archivo no existe, retorna un diccionario vacío.

5.2.3

```
def save_overrides(cat: str, edits: dict):  
    # Merge: Lo nuevo pisa lo anterior; nunca se borra el original  
    p = _path_for(cat)  
    data = load_overrides(cat)  
    for k, v in edits.items():  
        # normaliza key a str (guardamos por índice de oración)  
        data[str(k)] = v  
    with open(p, "w", encoding="utf-8") as f:  
        json.dump(data, f, ensure_ascii=False, indent=2)
```

Ilustración 26 Función: save_overrides(cat: str, edits: dict) - overrides.py

Propósito: Guarda los overrides para una categoría, fusionando con los existentes.

Comportamiento:

- Carga los overrides actuales (si existen).
- Actualiza el diccionario con los nuevos edits, normalizando las claves a string.
- Guarda el diccionario completo en el archivo, sobrescribiéndolo.

Nota: No elimina overrides anteriores que no estén en edits (merge).

5.2.4

```
def apply_overrides(selecciones: list, overrides: dict) -> list:  
    """Devuelve lista de listas de imágenes (final + overrides)."""  
    out = []  
    for idx, item in enumerate(selecciones):  
        base = (item.get("secuencia_final") or item.get("elecciones") or [])[:]  
        if str(idx) in overrides:  
            ov = overrides[str(idx)]  
            # Asegura misma longitud que tokens (rellena con None)  
            tokens = item.get("tokens", [])  
            base = (ov + [None]*len(tokens))[:len(tokens)]  
        out.append(base)  
    return out
```

Propósito: Aplica los overrides a una lista de selecciones (secuencias originales).

Parámetros:

- selecciones: Lista de diccionarios con las secuencias originales.

- overrides: Diccionario de overrides (índice -> lista de imágenes).

Retorno: Una lista de listas de imágenes, donde cada sublista es la secuencia de imágenes para una oración, con los overrides aplicados.

Lógica de aplicación:

1. Para cada oración (por índice), toma la secuencia base (secuencia_final o elecciones).
2. Si existe un override para ese índice, lo usa y ajusta la longitud para que coincida con la cantidad de tokens, rellenando con None si es necesario.
3. Si no hay override, usa la secuencia base.

5.3 Integración con la Aplicación

5.3.1 En routes.py (Categoría Detalle)

En la función `categoria_detalle`, cuando se maneja el método POST (descarga), se aplican overrides temporales (enviados por el formulario) pero no se persisten. El módulo `overrides.py` se utiliza para cargar y guardar overrides persistentes, aunque en la lógica actual de `routes.py` no se llama directamente a `load_overrides` o `save_overrides`. Sin embargo, la funcionalidad está disponible para ser integrada cuando se requiera persistencia.

5.3.2 Configuración Relacionada

En `config.py`, la variable `PERSIST_OVERRIDES` controla si se aplican los overrides guardados al recargar. Sin embargo, en el código actual de `routes.py` no se utiliza `apply_overrides` de manera persistente. En su lugar, se aplican overrides temporales (enviados por el cliente) en la función `_zip_categoria`.

5.4 Flujo de Datos

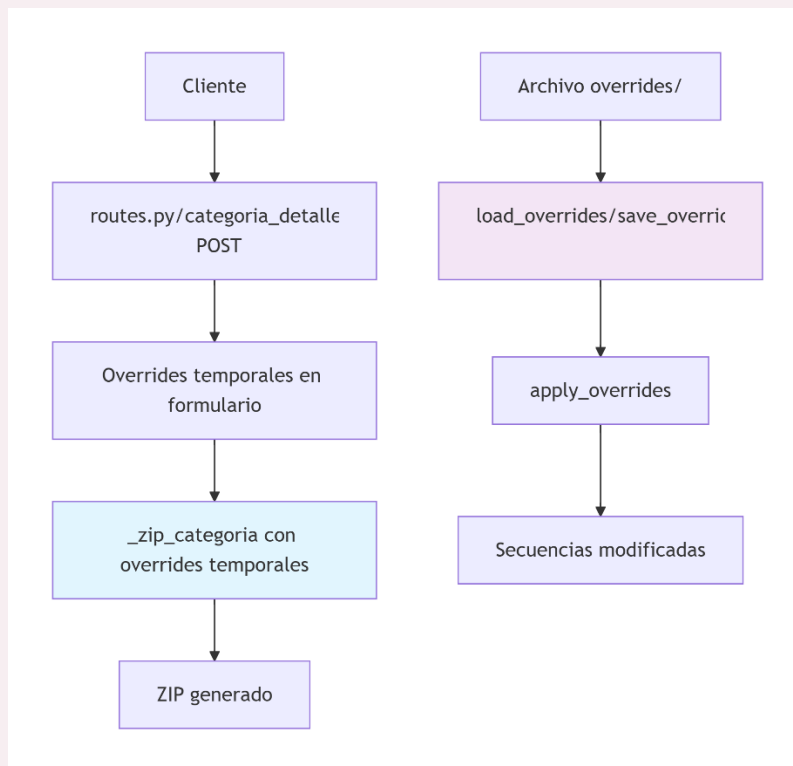


Ilustración 27 Flujo de Datos Overrides

Nota: En el estado actual, los overrides persistentes no se están utilizando en las rutas. Para integrarlos, se debería modificar `routes.py` para que, al cargar una categoría, llame a `load_overrides` y luego a `apply_overrides` si `PERSIST_OVERRIDES` es `True`.

6.Fachada para el Servicio BETO – `pictoweb/beto_service.py`

6.1 Vision general

El módulo `beto_service.py` actúa como **fachada (facade)** que abstrae el modo de inferencia (local o remoto) del modelo BETO. Proporciona una interfaz unificada para obtener pictogramas candidatos a partir de texto, delegando la implementación específica a los módulos correspondientes.

6.2 Funciones Principales

```
# pictoweb/beto_service.py
from flask import current_app

def infer_remote(text: str) -> dict:
    try:
        import requests
    except ImportError as e:
        raise RuntimeError("El modo remoto requiere instalar 'requests' (pip install requests).") from e

    url = current_app.config.get("BETO_REMOTE_URL")
    if not url:
        raise RuntimeError("BETO_REMOTE_URL no configurado.")
    r = requests.post(url, json={"text": text}, timeout=120)
    r.raise_for_status()
    return r.json()

def infer_local(text: str, dominio: str | None = None) -> dict:
    from . import beto_local
    return beto_local.infer_sentence(text, dominio=dominio)

def infer_pictos_for_text(text: str, dominio: str | None = None) -> dict:
    mode = (current_app.config.get("BETO_INFERENCE_MODE") or "local").lower()
    if mode == "local":
        return infer_local(text, dominio=dominio)
    return infer_remote(text)
```

Ilustración 28 Funciones Principales - beto_service.py

6.2.1 infer_remote(text: str) -> dict

Propósito: Envía una solicitud a un servicio remoto de BETO para obtener inferencias.

Parámetros:

- text: Texto a procesar.

Comportamiento:

- Verifica la disponibilidad de la librería requests.
- Obtiene la URL del servicio desde la configuración.
- Realiza una petición POST con timeout de 120 segundos.
- Retorna la respuesta JSON del servicio.

Nota: Esta función no se utiliza actualmente ya que BETO_INFERENCE_MODE = "local".

6.2.2 infer_local(text: str, dominio: str | None = None) -> dict

Propósito: Utiliza la implementación local del modelo BETO para generar inferencias.

Parámetros:

- text: Texto a procesar.
- dominio: Categoría para filtrar candidatos (salud, educación, cultura).

Retorno: Resultado de `beto_local.infer_sentence()`.

6.2.3 `infer_pictos_for_text(text: str, dominio: str | None = None) -> dict`

Propósito: Función principal que selecciona el modo de inferencia basado en configuración.

Lógica de Selección:

- Lee `BETO_INFERENCE_MODE` de la configuración (default: "local").
- Ejecuta inferencia local o remota según corresponda.

6.3 Integración con la Aplicación

6.3.1 Uso en `routes.py`

```
# En crear_picto_secuencia()
```

```
result = infer_pictos_for_text(frase, dominio=None)
```

```
# En api_reinfer()
```

```
result = infer_pictos_for_text(frase, dominio=dominio)
```

```
# En api_reinfer_token()
```

```
result = infer_pictos_for_text(frase, dominio=dominio)
```

6.3.2 Configuración Relacionada en `config.py`

- `BETO_INFERENCE_MODE = "local"` # Siempre local en configuración actual
- `BETO_MODEL_NAME = "dccuchile/bert-base-spanish-wwm-cased"`

6.4 Flujo de Datos

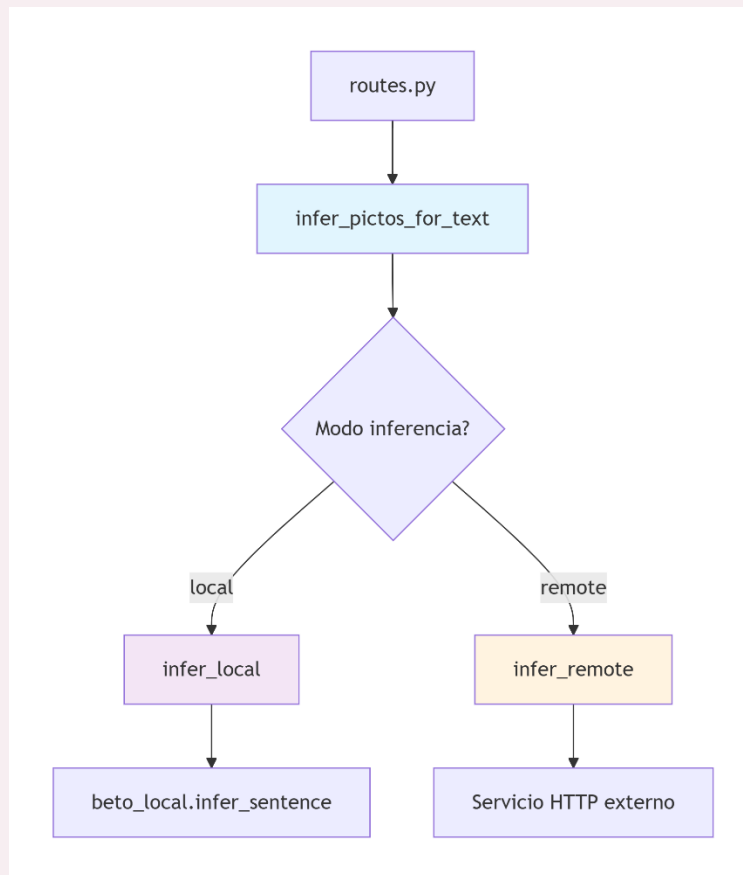


Ilustración 29 Flujo de Datos - beto_service.py

6.5 Dependencias

- `flask.current_app`: Para acceder a configuración.
- `beto_local`: Implementación local del modelo (importada dinámicamente).
- `requests`: Solo requerida para modo remoto (no usado actualmente).

7. Referencia Externa – pictoweb/beto_local.py

7.1 Visión General

El módulo `beto_local.py` contiene la implementación completa del pipeline de inferencia local del modelo BETO. Debido a su extensión y complejidad, este módulo cuenta con **documentación técnica independiente** que detalla:

- Carga y gestión del modelo BERT en español
- Generación de embeddings para texto y pictogramas

- Sistema de cacheo de embeddings
- Algoritmos de búsqueda semiestructurada
- Procesamiento de tokens y lematización
- Integración con el sistema de candidatos por categoría

7.2 Puntos de Integración con la Aplicación

7.2.1 Función Principal

Llamada desde beto_service.py

def infer_sentence(text: str, dominio: str | None = None) -> dict

Parámetros de Entrada:

- text: Cadena de texto a procesar
- dominio: Categoría para filtrar candidatos (opcional)

Estructura de Salida:

```
{
  "tokens": ["lista", "de", "tokens"],
  "sugerida": ["imagen1.jpg", "imagen2.jpg", null],
  "candidatos_por_token": {
    "token1": ["img1.jpg", "img2.jpg"],
    "token2": ["img3.jpg", "img4.jpg"]
  }
}
```

7.2.2 Dependencias Configuradas en config.py

BETO_MODEL_NAME = "dccuchile/bert-base-spanish-wwm-cased"

BETO_CACHE_EMB = "ruta/cache/emb_pictos_cache.npy"

BETO_FORCE_CPU = False

7.3 Referencia Cruzada

Para detalles completos de implementación, algoritmos y optimizaciones, consultar el **Manual Técnico de beto_local.py** (Manual técnico: Notebook de Recomendación de Pictogramas para Comunicación Aumentativa).

8.Fachada para el Servicio CLIP – pictoweb/clip_service.py

8.1 Visión General

El módulo clip_service.py actúa como fachada para integrar el modelo CLIP multimodal (texto+imagen) que optimiza la selección de pictogramas basándose en similitud semántica entre descripciones textuales y características visuales.

8.2 Integración con la Aplicación

8.2.1 Uso en routes.py – API CLIP

```
@bp.post("/api/clip/assemble")
def api_clip_assemble():
    # ... validación de payload
    from .clip_service import assemble_clip
    result = assemble_clip(oracion, tokens, candidatos)
    return jsonify({"ok": True, "result": result})
```

Ilustración 30 Función donde se utiliza CLIP

8.2.2 Configuración Relacionada __init__.py

```
app.config.setdefault("CLIP_TEXT_MODEL", "sentence-transformers/clip-ViT-B-32-multilingual-v1")
app.config.setdefault("CLIP_IMAGE_MODEL", "clip-ViT-B-32")
app.config.setdefault("CLIP_FORCE_CPU", False)
app.config.setdefault("CLIP_REMOTE_URL", None) # Para futuro uso remoto
```

Ilustración 31 Llamada a los modelos CLIP en __init__.py

8.3 Flujo de Procesamiento CLIP

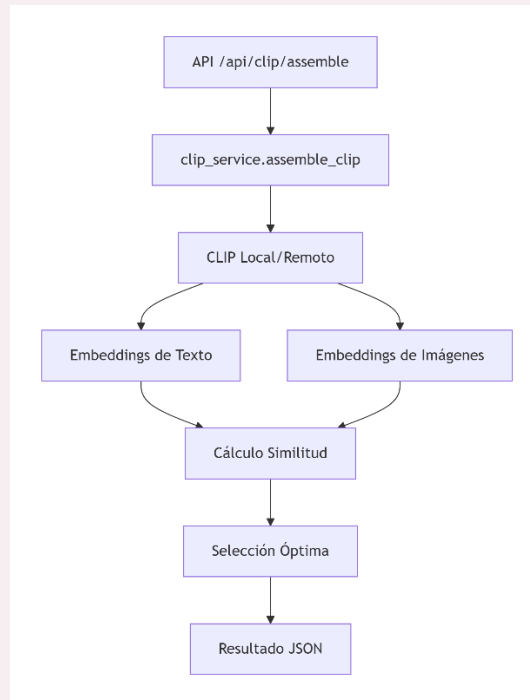


Ilustración 32 Flujo de Procesamiento – CLIP

8.4 Casos de Uso

1. **Refinamiento post-BETO:** Después de que BETO genera candidatos, CLIP puede optimizar la selección final.
2. **Búsqueda multimodal:** Cuando se necesita considerar tanto el significado textual como la similitud visual.
3. **Desambiguación:** Para tokens polisémicos donde el contexto visual es importante.

9. Referencia Externa – [pictoweb/clip_local.py](#)

9.1 Visión General

`clip_local.py` contiene la implementación local del modelo CLIP, incluyendo:

- Carga de modelos preentrenados de texto e imagen
- Generación de embeddings multimodales
- Cálculo de similitud coseno entre textos e imágenes

De igual forma que `beto_local.py`, se desarrolló un **manual técnico de `clip_local.py`** (Manual técnico: Notebook de Selección de Pictogramas Multimodal con CLIP)

10. Templates del sistema

10.1 base.html – Layout Principal

Propósito y Contexto de Uso

base.html es el **template base** que define la estructura común para todas las páginas de PICTOSEQ. Implementa el patrón de **herencia de templates** de Jinja2, proporcionando un layout consistente con navegación, estilos y scripts compartidos que todos los demás templates extienden.

10.2 home.html – Página de Inicio

Propósito y Contexto de Uso

home.html es la **página de aterrizaje (landing page)** principal de PICTOSEQ que presenta la aplicación y ofrece acceso rápido a sus funcionalidades principales. Extiende base.html y está diseñada para:

1. **Presentar el proyecto** y su propósito de manera atractiva
2. **Proporcionar llamadas a la acción (CTAs)** claras hacia las funcionalidades clave
3. **Mostrar las categorías disponibles** mediante accesos rápidos visuales
4. **Establecer la identidad visual** del sistema con elementos gráficos personalizados

Ruta asociada: / (raíz del sitio)

Contexto de uso:

- Punto de entrada para nuevos usuarios
- Página principal de navegación
- Vista de presentación del sistema

10.3 acerca_de.html – Página “Acerca de”

Propósito y Contexto de Uso

acerca_de.html es la página de **información institucional** que detalla los fundamentos, objetivos y contexto del proyecto PICTOSEQ. Extiende base.html y está diseñada para:

1. **Explicar el propósito y necesidad** del sistema en contextos educativos, culturales y de salud

2. **Describir la problemática actual** que el proyecto busca resolver
3. **Presentar la propuesta tecnológica** basada en IA y modelos de lenguaje natural
4. **Proporcionar una visión general** del funcionamiento del sistema
5. **Destacar los ámbitos de aplicación** y beneficios del proyecto

Ruta asociada: /acerca-de

Contexto de uso:

- Información para usuarios, colaboradores e interesados en el proyecto
- Contexto académico e institucional del desarrollo
- Explicación de la tecnología y metodología utilizada

10.4 manual_usuario.html – Página “Manual de Usuario”

Propósito y Contexto de Uso

manual_usuario.html es la página de **documentación y recursos de usuario** que integra un visor de PDF interactivo para consultar el manual de usuario directamente en el navegador. Extiende base.html y está diseñada para:

1. **Proporcionar acceso inmediato** al manual de usuario mediante un visor PDF embebido
2. **Ofrecer descarga directa** del manual en formato PDF para uso offline
3. **Implementar medidas de seguridad** que prevengan la descarga no autorizada del PDF desde el visor
4. **Garantizar una experiencia responsiva** que funcione en dispositivos móviles y desktop

Ruta asociada: /manual-usuario

Contexto de uso:

- Acceso rápido a la documentación del sistema para usuarios nuevos y existentes
- Recursos de aprendizaje y referencia para el uso de todas las funcionalidades
- Punto centralizado de documentación oficial del proyecto

Características técnicas destacadas:

- Visor PDF embebido con iframe
- Escudos transparentes (overlays) que bloquean la barra de herramientas del visor para prevenir descargas no autorizadas
- Diseño responsivo que adapta la altura del visor según el dispositivo
- Enlace directo de descarga del archivo PDF

10.5 categoria.html – Hub de Categorías

Propósito y Contexto de Uso

categoria.html es la página que actúa como **hub de categorías**, mostrando las tres categorías disponibles (salud, educación, cultura) con tarjetas visuales de acceso. Extiende base.html y está diseñada para:

1. **Presentar de forma visual** las categorías de secuencias disponibles en el sistema
2. **Mostrar el conteo de secuencias** en cada categoría (pasado desde el backend mediante la variable counts)
3. **Proporcionar acceso directo** a la vista detallada de cada categoría mediante botones "Entrar"
4. **Ofrecer una experiencia visual atractiva** con animaciones y diseño responsivo

Ruta asociada: /categorias

Contexto de uso:

- Página de selección de categoría para usuarios que desean explorar secuencias existentes
- Vista de resumen que muestra la cantidad de secuencias en cada categoría
- Punto de partida para acceder a las funcionalidades de visualización, edición y descarga por categoría

Datos recibidos del backend:

- counts: Diccionario con las claves 'salud', 'educacion', 'cultura' y el número de secuencias en cada una

Características de diseño:

- Grid responsivo con tarjetas (cards) para cada categoría
- Imágenes ilustrativas para cada categoría (tarjeta_salud.png, tar_educacion.png, tarjeta_cultura.png)
- Animaciones de entrada (AOS) con retrasos escalonados
- Estilo de tarjeta consistente con el diseño del sistema

10.6 categoria_detalle.html – Detalle de Categoría con Carrusel

Propósito y Contexto de Uso

categoria_detalle.html es el template más complejo e interactivo del sistema, diseñado para **visualizar, editar y descargar secuencias de pictogramas** por categoría. Extiende base.html y proporciona:

1. **Visualización en carrusel:** Muestra cada oración con su secuencia actual en un carrusel interactivo automático
2. **Tabla de gestión:** Lista todas las secuencias de la categoría con opciones de selección para descarga
3. **Sistema de edición dual:**
 - **Modo 1:** Modificar secuencias (reemplazar pictogramas individuales)
 - **Modo 2:** Sustituir/eliminar palabras (editar tokens y regenerar candidatos)
4. **Descarga selectiva:** Permite seleccionar secuencias específicas con overrides aplicados
5. **Persistencia temporal:** Guarda cambios en tiempo real sin afectar los datos originales

Ruta asociada: /categoria/<cat> (salud, educación, cultura)

Contexto de uso:

- Vista principal para explorar y trabajar con secuencias existentes
- Interfaz para personalizar secuencias antes de descargar
- Punto de integración entre visualización, edición y exportación

Datos recibidos del backend:

- cat: Categoría actual (salud, educación, cultura)

- selecciones: Lista de secuencias completas con oración, tokens e imágenes
- effective: Secuencias con overrides aplicados (si los hay)
- candidatos_map: Mapeo de candidatos por oración y token para edición

Características técnicas destacadas:

- Carrusel Swiper.js con autoplay y navegación
- Sistema de edición en dos modos con UI dinámica
- Integración con API /api/reinfer_token para regeneración en tiempo real
- Manejo de overrides de imágenes y tokens en objetos JSON
- Validación de stopwords y palabras sin pictogramas
- Interfaz responsiva con tablas y grids adaptables

10.7 crear_picto.html – Creación de Secuencias Personalizadas

Propósito y Contexto de Uso

crear_picto.html es el template para la **creación personalizada de secuencias de pictogramas** a partir de texto libre, integrando tanto el modelo BETO para generación inicial como CLIP para optimización. Extiende base.html y proporciona:

1. **Generación de secuencias:** A partir de texto libre usando el modelo BETO en español
2. **Selección interactiva:** Permite seleccionar pictogramas alternativos para cada token
3. **Optimización con CLIP:** Integración del modelo multimodal para "armar" la mejor secuencia
4. **Gestión de carrito:** Sistema para acumular múltiples secuencias antes de descargar
5. **Interfaz de carga:** Overlays y spinners para operaciones asíncronas con IA

Ruta asociada: /crear

Contexto de uso:

- Creación de secuencias personalizadas desde cero
- Experimentación con diferentes combinaciones de pictogramas
- Acumulación de múltiples secuencias para descarga en lote

- Comparación entre sugerencias BETO y optimizaciones CLIP

Datos recibidos del backend:

- frase: Texto ingresado por el usuario
- tokens: Lista de tokens identificados
- sugerida: Lista de pictogramas sugeridos por BETO
- candidatos_map: Mapeo de candidatos alternativos por token
- saved_list: Secuencias acumuladas en el carrito (persistente en formulario)
- error: Mensaje de error en caso de fallo

Características técnicas destacadas:

- Sistema de carrito en memoria del navegador con persistencia en campo oculto
- Integración con APIs /api/clip/assemble para optimización con CLIP
- Overlay de carga global para operaciones asíncronas
- Selección visual de pictogramas con estado "seleccionado"
- Manejo de estado local (window.__currentSeq) para la secuencia en edición
- Validación de formularios y manejo de teclas (Enter para enviar)

11. Consideraciones finales del sistema

Fortalezas Arquitectónicas

1. **Separación clara de responsabilidades:** MVC bien implementado
2. **Cacheo estratégico:** Multi-nivel (embeddings, datos JSON, templates)
3. **Extensibilidad modular:** Facilita añadir nuevas categorías y modelos
4. **Interfaz progresiva:** Mejora experiencia sin sacrificar funcionalidad base
5. **Persistencia flexible:** Overrides opcionales, datos originales inmutables

Manual Técnico Completado

- **Configuración y Arranque:** __init__.py, config.py, run.py

- **Capa Web y Rutas:** routes.py (controlador principal)
- **Capa de Datos:** data_access.py, overrides.py
- **Servicios de IA:** beto_service.py, clip_service.py (referencia a beto_local.py, clip_local.py)
- **Templates:** Todos los archivos en templates/

Diagramas sitio PICTOSEQ

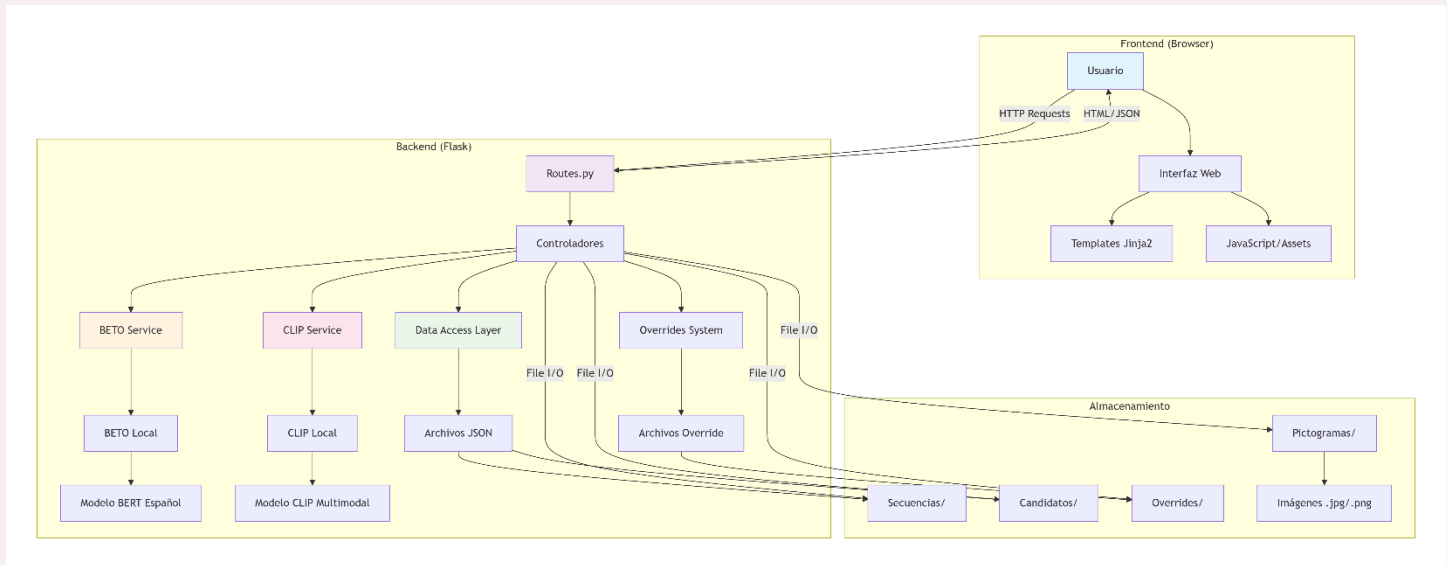


Ilustración 33 DIAGRAMA PICTOSEQ 1

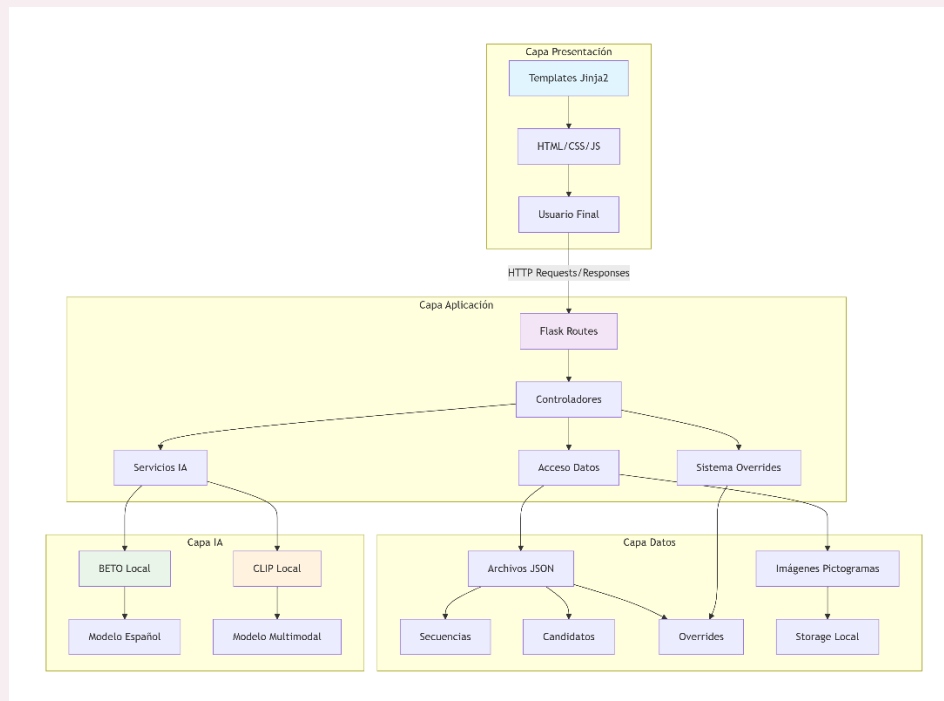


Ilustración 34 DIAGRAMA PICTOSEQ 2

MANUAL TÉCNICO PICTOSEQ v1.0 COMPLETADO

Este documento cubre la arquitectura completa, módulos, templates y consideraciones técnicas del sistema PICTOSEQ. Sirve como referencia para desarrolladores, mantenedores y futuras extensiones del proyecto.